HOSTEL MANAGEMENT SYSTEM

SECURITY TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared test plan and test cases; execution results pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Performance testing report supplied as format reference; system implementation details not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	Security Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Planned cases documented; no execution evidence supplied
Repository destination	reports/security-testing-report.docx on the report branch

1. Introduction

This report defines a security-testing approach for the Hostel Management System (HMS) described in the supplied performance-testing report. It focuses on protecting student and staff accounts, hostel applications, room and bed allocations, complaints, fee records, visitor information, notices, and administrative reports. The actual application, source code, deployment, and test evidence were not provided; this document is a test plan and does not claim that tests have been executed or that the system is secure.

2. Objectives

• Verify that each user can access only the functions and records allowed for their role.

• Check authentication, session handling, account recovery, and resistance to common input attacks.

• Protect sensitive student, payment, visitor, and accommodation data in transit and at rest where applicable.

• Check that security-relevant actions are logged without exposing passwords, tokens, or unnecessary personal data.

• Record reproducible evidence, findings, severity, and remediation status for each executed test.

3. Scope

Included area	Security activity
Authentication and sessions	Check login behavior, logout, session expiry, cookie attributes, and account recovery.
Role access control	Test student, warden, and administrator permissions; verify deny-by-default behavior.
Student and application records	Attempt unauthorized viewing or modification through UI and direct requests.
Room / bed allotment	Test unauthorized allotment changes, conflict manipulation, and record ownership.
Complaints and notices	Check visibility rules, staff-only actions, and input handling.
Fees and payments	Check access to fee data, verification actions, and tampering of submitted values.
Visitors	Check access restrictions and protection of visitor details.
Reports and exports	Test administrative access, filters, exports, and leakage of unrelated records.
Input handling and errors	Check injection, cross-site scripting, request forgery where relevant, and safe error responses.
Audit and configuration	Review security event logging, transport security, secrets/configuration, and dependency exposure where test access permits.

Out of scope: source-code audit, infrastructure penetration testing, production certification, and third-party service assessment unless separately authorized and provided. Testing must use a controlled non-production environment.

4. Test Environment and Preconditions

Environment item	Value to record before testing
Application build / commit	_____
Application URL / deployment type	_____
Framework, OS, and server details	_____
Database engine and version	_____
Test tools and versions	_____
Roles and test accounts	_____
Network / TLS configuration	_____
Test date and tester	_____

Preconditions

- Obtain written authorization and agree test scope, time window, accounts, and prohibited actions.
- Use an isolated test environment and synthetic records; do not test live student or financial data.
- Create least-privilege accounts for student, warden, and administrator roles, plus records owned by multiple test users.
- Back up or reset test data before state-changing tests; verify recovery steps.
- Capture request/response evidence with secrets and personal data redacted; preserve relevant application and audit logs.

5. Security Test Cases

All cases below are proposed and marked Not run. Confirm applicable roles and features against the implementation before execution. Use benign test payloads and stop if a test affects data outside the approved environment.

ID	Scenario	Procedure / expected result	Status
ST-01	Role access matrix	For each role, request permitted and restricted pages/actions. Denied actions return an appropriate denial and do not expose protected data.	Not run
ST-02	Record ownership	As one student, request another student's application, complaint, fee, or visitor record by changing identifiers. Access is denied and no data is disclosed.	Not run
ST-03	Administrative endpoints	Call staff/admin actions as a student and unauthenticated user. No protected action succeeds.	Not run
ST-04	Direct object reference	Modify record IDs in URLs and request bodies for rooms, applications, complaints, and fees. Server enforces authorization on every request.	Not run
ST-05	Login protections	Test invalid credentials, account enumeration clues, and repeated attempts within the agreed safe limit. Responses avoid revealing account existence; throttling/controls behave as specified.	Not run
ST-06	Session logout and expiry	Authenticate, log out or wait for configured expiry, then replay a prior session token. The token no longer grants access.	Not run
ST-07	Session cookie settings	Inspect cookies over HTTPS. Secure, HttpOnly, and suitable SameSite attributes are set where applicable; session identifiers rotate after authentication.	Not run
ST-08	Password recovery	Request recovery for valid and invalid accounts; inspect flow and tokens. Responses do not disclose account status; tokens are single-use and expire.	Not run
ST-09	Input injection	Submit harmless SQL/NoSQL metacharacter and query-manipulation strings in search and form fields. Inputs are safely handled; no unauthorized data or database errors appear.	Not run
ST-10	Stored/reflected XSS	Submit benign markup/script markers in complaints, notices, and profile fields. Output is encoded; no script executes in another user's browser.	Not run
ST-11	Request forgery	For state-changing actions, verify CSRF defenses or equivalent origin protections. Cross-site requests without valid protection do not change records.	Not run
ST-12	Parameter tampering	Change role, fee amount/status, student ID, allotment status, or approval fields in requests. Server-side validation rejects unauthorized changes.	Not run
ST-13	Sensitive data exposure	Inspect pages, API responses, exports, and errors for passwords, tokens, unnecessary personal data, and other users' records. Only authorized, necessary fields are returned.	Not run
ST-14	Transport security	Verify login and authenticated traffic use HTTPS in the target environment and do not downgrade or send credentials over cleartext.	Not run
ST-15	File upload / export	If uploads or exports exist, test file type/size validation and access restrictions with benign files. Files cannot execute or be retrieved by unauthorized users.	Not run
ST-16	Audit trail	Perform representative login, failed login, approval, allotment, fee verification, and role changes. Logs record actor/action/time and omit secrets.	Not run
ST-17	Error handling	Trigger invalid identifiers and malformed but benign inputs. Responses do not expose stack traces, SQL, filesystem paths, or secrets.	Not run
ST-18	Configuration review	Review test deployment for debug mode, default credentials, exposed admin/debug endpoints, and committed secrets. Record evidence and affected build.	Not run

6. Severity and Acceptance Criteria

Use the project-approved severity scheme. If none exists, record Critical / High / Medium / Low with rationale based on impact, exploitability, affected data, and required access. A security pass/fail decision requires executed tests, retained evidence, and agreed acceptance thresholds.

- No unauthorized role or user can read or modify protected records.
- No unresolved Critical or High findings remain before release, unless formally accepted by the responsible owner.
- Authentication and session controls behave as documented and protect authenticated access.
- Sensitive data is not exposed through responses, exports, logs, or error messages beyond authorized need.
- Security events needed for investigation are logged with sufficient context while excluding credentials and tokens.
- All findings have a severity, evidence, remediation owner, and retest outcome.

7. Results and Measurements

No application access, security scan output, test execution evidence, or implementation details were supplied. Therefore, this section contains no pass/fail claims. Complete it after authorized execution.

Test ID	Outcome	Evidence reference	Severity / notes
ST-01 to ST-06	Pending	_____	_____
ST-07 to ST-12	Pending	_____	_____
ST-13 to ST-18	Pending	_____	_____

7.1 Defect and Observation Log

ID	Finding / evidence	Severity	Owner	Status
SEC-01	To be completed after execution	TBD	TBD	Open
SEC-02	Add one row for each distinct finding	TBD	TBD	Open

8. Execution Notes and Report Limits

This document is a proposed security test plan based on the project name, institution, and functional areas listed in the supplied performance-testing report. The test cases may need adjustment after reviewing the actual HMS implementation and requirements. “Not run” means no test result is asserted. It is not evidence of a vulnerability or of security compliance.

9. Sign-off

Role	Name	Signature / date
Tester	_____	_____
Project supervisor	_____	_____
System owner / approver	_____	_____